



Fase 4 — Pipeline ETL (CSV → MySQL)

Nesta fase criei o pipeline que lê os CSVs gerados na fase anterior, faz uma limpeza básica e carrega os dados no banco MySQL.

A ideia é ter um passo automático entre “arquivos brutos” e “banco pronto para consultar”, sem precisar importar nada na mão.



Objetivo da Fase

- Ler os 7 CSVs de `data/raw/`
 - Fazer uma limpeza básica (espaços, valores vazios, tipos)
 - Carregar tudo no MySQL respeitando a ordem das chaves estrangeiras
 - Poder rodar o script mais de uma vez sem duplicar dados
-



Tecnologias Utilizadas

- **Python 3.x**
 - **Pandas** — leitura dos CSVs e limpeza
 - **SQLAlchemy** — conexão e carga no banco
 - **mysql-connector-python** — driver do MySQL
 - **python-dotenv** — configuração do banco via arquivo `.env`
-



Ordem de Carga

As tabelas são carregadas na ordem que respeita as chaves estrangeiras. Primeiro as que não dependem de ninguém, depois as que referenciam as anteriores:

1. `clientes`
 2. `fornecedores`
 3. `empresas`
 4. `movimentacoes_bancarias`
 5. `documentos_fiscais`
 6. `contas_financeiras`
 7. `conciliacoes`
-



Limpeza Aplicada

A limpeza é simples de propósito — o foco desta fase é a carga:

- **Espaços sobrando** nos textos são removidos (`strip`)

- **Strings vazias** viram `NULL`
- **FKs opcionais** (`id_cliente` , `id_fornecedor`) são convertidas para inteiro nulo, para não entrarem como texto vazio

Notas Fiscais Duplicadas

Aqui apareceu um ponto interessante. Na Fase 3 eu gerei de propósito algumas notas fiscais com a **mesma chave de acesso** (duplicatas), mas a tabela `documentos_fiscais` tem `UNIQUE` na `chave_acesso` . Ou seja, o banco não deixa gravar as duas.

Em vez de deixar o script quebrar, decidi tratar isso: guardo a **primeira ocorrência** de cada chave e mando as repetidas para um arquivo à parte:

```
data/processed/documentos_duplicados.csv
```

Assim nenhuma linha se perde silenciosamente — as duplicatas ficam separadas para conferir depois, e o restante é carregado normalmente.

Rodar Mais de Uma Vez

Antes de carregar, o script dá `TRUNCATE` nas tabelas (na ordem inversa, com as checagens de FK desativadas por um instante). Isso deixa o processo repetível: se eu rodar de novo, o banco não acumula dados duplicados.

Como Executar

1. Configurar o banco

Copie o `.env.example` para `.env` e ajuste com os seus dados:

```
cp .env.example .env
```

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=fiscalaudit
DB_USER=root
DB_PASSWORD=sua_senha
```

2. Criar as tabelas (se ainda não criou)

```
mysql -u root -p fiscalaudit < sql/01_create_tables.sql
```

3. Rodar o ETL

```
python src/etl/carregar_dados.py
```

Saída esperada

Carregando dados no MySQL...

- ✓ clientes: 50 registros carregados
- ✓ fornecedores: 40 registros carregados
- ✓ empresas: 10 registros carregados
- ✓ movimentacoes_bancarias: 500 registros carregados
11 documento(s) com chave duplicada separados em documentos_duplicados.csv
- ✓ documentos_fiscais: 289 registros carregados
- ✓ contas_financeiras: 400 registros carregados
- ✓ conciliacoes: 112 registros carregados

Concluído. 1401 registros no total.



Próximos Passos

Com os dados no banco, a próxima fase é o **motor de regras de auditoria** (Fase 5), que vai cruzar as tabelas e identificar as inconsistências que foram plantadas na Fase 3.